I genuinely think this is worth pursuing because it follows the same philosophy as your roommate's projects: **it's solving an operational problem**, not showcasing a technology. The difference is that it's solving a consumer productivity problem instead of an enterprise IT problem.

I'd actually recommend documenting it like a startup would.

Project Vision

Working Title

DocVault *(temporary)*

> A cross-platform personal document management system that automatically captures, organizes, synchronizes, and securely shares important documents across all of a user's devices.

Problem Statement

Background

In today's digital world, important documents arrive from multiple sources:

- Gmail attachments
- WhatsApp documents
- Browser downloads
- College portals
- Government websites
- Google Drive
- OneDrive
- Mobile downloads
- Scanned copies

Because these documents originate from different devices and applications, users often lose track of where they are stored.

The result is duplicated files, inconsistent organization, wasted time searching for documents, and repeated manual sharing with friends, family, recruiters, or institutions.

Core Problem

> There is currently no unified personal document management system that automatically captures important documents from multiple devices, keeps them synchronized, intelligently organizes them, and enables secure, permission-based access from anywhere.

Existing Workflow

```
Receive document

↓

Download somewhere

↓

Forget where it was saved

↓

Need it months later

↓

Search Gmail

↓

Search Downloads

↓

Search WhatsApp

↓

Search Google Drive

↓

Search Laptop

↓

Finally find it (or don't)
```

Proposed Solution

Create a cross-platform document management ecosystem that automatically captures, indexes, synchronizes, and organizes important documents while making them instantly searchable and securely shareable across all devices.

Objectives

The application should

- Eliminate scattered document storage.
- Reduce time spent searching for files.
- Synchronize documents across multiple devices.
- Preserve document history.
- Enable secure document sharing.
- Automatically organize documents with metadata.
- Make retrieval nearly instantaneous.

Target Users

Primary

- College students
- Professionals
- Job applicants
- Freelancers

Secondary

- Families
- Small businesses
- Teachers
- Consultants

Pain Points Solved

Before

"I know I downloaded this somewhere."

"I think it's in Gmail."

"I sent it to myself on WhatsApp."

"I have three copies of the same PDF."

"I'll organize it later."

"I can't find my internship offer letter."

After

Open app

↓

Search

```
Internship Offer
```

↓

Found in under one second.

Unique Selling Proposition

Unlike cloud storage platforms that focus on storing files,

DocVault focuses on managing important documents throughout their lifecycle.

The emphasis is on

- capture
- organization
- synchronization
- retrieval
- secure sharing

rather than simple storage.

Functional Requirements

Document Capture

- Desktop download monitoring

- Mobile Share Sheet integration
- Manual upload
- Drag-and-drop upload
- Camera scanning
- Bulk upload

Organization

- Categories
- Tags
- Metadata
- Folder support
- Collections
- Favorites

Retrieval

- Full-text search
- OCR search
- Metadata filters
- Tag search
- Date search
- Source search

Synchronization

- Laptop ↔ Cloud
- Phone ↔ Cloud
- Tablet ↔ Cloud
- Automatic conflict resolution
- Background synchronization

Sharing

- Share individual documents
- Share folders
- Share collections
- Permission-based access

- Expiring links
- Read-only access
- Edit access

Security

- Authentication
- Role-based permissions
- Encryption
- Audit logs
- Device management

Non-functional Requirements

- High availability
- Fast search (<1 sec)
- Cross-platform
- Secure
- Scalable
- Responsive
- Reliable synchronization

Technologies You'll Demonstrate

Frontend

- React
- Tailwind CSS

Skills demonstrated

- Component architecture
- State management
- Responsive UI
- File uploads
- Search interfaces

Backend

FastAPI

Skills

- REST APIs
- Authentication
- File processing
- Background workers
- Business logic
- Permission management

Database Design

Likely PostgreSQL

Skills

- Relational modelling
- Normalization
- Indexing
- Transactions
- Metadata storage

Cloud Storage

Object storage (S3-compatible, Azure Blob, etc.)

Skills

- File storage architecture
- Signed URLs
- Efficient uploads/downloads
- Storage abstraction

Desktop Development

Background application

Responsibilities

- Monitor Downloads

- Detect new documents
- Prompt user
- Upload automatically
- Sync changes

Skills

- File system events
- Background services
- Native notifications

Mobile Development

Android application

Responsibilities

- Share files directly
- Camera scan
- Offline access
- Notifications
- Sync

Skills

- Mobile storage
- Share intents
- Background sync
- Secure authentication

Search Engine

Could use PostgreSQL Full-Text Search initially, with room to adopt Elasticsearch/OpenSearch later.

Skills

- Indexing
- Ranking
- Fast retrieval

OCR

Extract searchable text from

- PDFs
- Images
- Scanned documents

Skills

- Document parsing
- Text extraction
- Search indexing

Synchronization Engine

Probably the most impressive backend component.

Responsibilities

- Detect changes
- Sync devices
- Handle conflicts
- Version history
- Retry failed syncs

Skills

- Distributed systems concepts
- Event processing
- Conflict resolution
- Caching

Authentication

JWT

Refresh Tokens

Sessions

Device management

Skills

- Security
- Authorization

- Token lifecycle

Permission Management

Users

↓

Collections

↓

Shared Users

↓

Roles

Viewer

Editor

Owner

Admin

Background Processing

Examples

- OCR
- Thumbnail generation
- Virus scanning
- Search indexing
- Sync jobs
- Notifications

Skills

- Queues
- Workers
- Async processing

Software Engineering Concepts Demonstrated

- Full-stack architecture
- Client-server communication
- Distributed synchronization
- Authentication & Authorization
- Object storage
- Relational database design
- Background job processing
- File indexing
- Search systems
- Event-driven programming
- Offline-first considerations
- API design
- Caching
- System scalability
- Secure file handling

Resume Value

Instead of saying:

> Built a document storage website.

You can honestly say:

> **Designed and developed a cross-platform personal document management system featuring automated document capture, intelligent metadata management, full-text search, cross-device synchronization, background file monitoring, secure cloud storage, and permission-based document sharing.**

That immediately sounds like a system with real engineering depth.

What makes this project different?

One thing I'd challenge you on before you write a single line of code is this: **define one feature that would make someone choose DocVault over simply creating a folder in Google Drive.**

At the moment, I think that differentiator is:

> **DocVault doesn't expect users to remember to organize their documents. It proactively captures documents from the places they naturally arrive (Downloads, Gmail, WhatsApp, browser, scanner), enriches them with metadata, synchronizes them across devices, and makes them instantly retrievable through a unified search interface.**

That sentence should become the guiding principle for every feature you add. If a feature doesn't strengthen that promise, it probably doesn't belong in the first version.